ASSIGNMENT 3: MONTE CARLO METHODS

Physics & Computational Methods

IMPORTANT INSTRUCTIONS

- All questions in this assignment are coding questions.
- **SUBMISSION FORMAT:**
  - Create one Notebook (.ipynb) file
  - Solve all four questions in this single notebook
  - Add clear markdown cells with question titles
  - Include comments in your code to explain it wherever needed
  - Upload the completed (.ipynb) file to Google Classroom

For queries contact us on Whatsapp.

Question 1: Two-Dimensional Random Walk

A particle starts at the origin $(0, 0)$ and takes 100 random steps on a 2D grid. At each step, it moves:

- Right $(+1, 0)$ with probability 1/4
- Left $(-1, 0)$ with probability 1/4
- Up $(0, +1)$ with probability 1/4
- Down $(0, -1)$ with probability 1/4

Run 500 independent random walks and analyze the spreading behavior.

Part (a): Simulate 500 random walks, each with 100 steps. For each walk, record the final position (x, y) and calculate the distance from origin $r = \sqrt{x^2 + y^2}$. Store these distances in an array called `final_distances`.
Part (b): Create a histogram with 25 bins showing the distribution of final distances from all 500 walks. Label the x-axis as "Distance from Origin" and the y-axis as "Frequency". Add a title "Random Walk: Distribution of Final Distances".
Part (c): Calculate the average of r^2 (the squared distances) and store it in a variable called `avg_r_squared`. Print the result with the message "Average r-squared = X

(theoretical value = 100)". Explain in 2-3 sentences why this matches the theoretical expectation.

Question 2: Buffon's Needle Problem

A needle of length (L) 0.5 units is dropped randomly on a floor with parallel lines spaced 1 unit apart. The needle's center position y from the nearest line is uniformly random in $[0, 0.5]$, and its angle θ is uniformly random in $[0, \pi]$. The needle crosses a line if: $y \leq 0.25 \sin(\theta)$.

Estimate the value of π using this random experiment.

Part (a): Drop 10,000 needles. For each needle, generate random y and θ, check if it crosses a line (store as 0 or 1), and count the number of crossings. Store the total count in a variable called `num_crossings`.
Part (b): Estimate π using the formula: $\pi_{\text{estimated}} = \frac{2 \cdot totalneedles \cdot L}{\text{num\_crossings}}$. Store this in a variable called `pi_estimate`. Print the result with the message "Estimated pi = X.XXXX (true value = 3.1416, error = Y%)".
Part (c): Create a line plot showing the running estimate of π as more needles are added (use increments of 100 needles up to 10,000). Label the x-axis as "Number of Needles" and the y-axis as "Estimated Pi". Add a horizontal dashed red line at the true value of π. Add a title "Convergence of Pi Estimate via Buffon's Needle".

Question 3: Radioactive Decay

A sample initially contains 1000 radioactive nuclei with decay constant $\lambda = 0.01$ (inverse time units). The number of nuclei remaining at time t follows: $N(t) = N_0 \exp(-\lambda t)$. The half-life is $T_{1/2} = \ln(2)/\lambda \approx 69.3$ time units.

Simulate individual decay events and compare to the analytical formula.

Part (a): Generate decay times for 1000 nuclei using inverse transform sampling: $t_{\text{decay}} = -\ln(U)/\lambda$ where $U \sim \text{Uniform}(0, 1)$. Store these times in an array called `decay_times`.
Part (b): At each time point $t = 0, 20, 40, 60, \ldots, 500$, count how many nuclei have **not** decayed yet (i.e., $t_{\text{decay}} > t$). Store these counts in a list called `remaining_nuclei`. Also compute the analytical curve: $N(t) = 1000 \exp(-0.01 \cdot t)$ for the same time points and store in a list called `analytical_N`.
Part (c): Create a plot with both the simulated data (plot as blue points) and the analytical curve (plot as a red line). Label the x-axis as "Time (units)" and the y-axis as "Number of Nuclei". Add a title "Radioactive Decay: Simulation vs Theory". Add a legend. Verify visually that they match closely.

Question 4: High-Dimensional Integration and Curse of Dimensionality

Estimate the integral $I_d = \int_{[0,1]^d} \exp\left(-\sum_{i=1}^{d} x_i\right) d\mathbf{x}$ using two methods for dimensions $d = 1, 2, 3, 5, 10$. The exact value is: $I_d = (1 - e^{-1})^d \approx (0.6321)^d$.

Compare grid-based integration (midpoint rule) with Monte Carlo integration under a fixed computational budget of 100,000 function evaluations.

Part (a): For each dimension d, implement:

- **Grid method:** Set $m = \lfloor 100000^{1/d} \rfloor$ grid points per dimension. Evaluate the integrand at all m^d grid points using midpoints $(k + 0.5)/m$. Estimate the integral as the average function value. Store results in lists: `grid_m`, `grid_evals` ($= m^d$), `grid_estimates`, `grid_errors`.
- **Monte Carlo:** Generate 100,000 random points uniformly in $[0, 1]^d$ and estimate the integral as the average function value. Store results in lists: `mc_estimates`, `mc_errors`.

Part (b): Create two plots:

1. **Plot 1 (log scale):** x-axis = dimension d, y-axis = number of grid evaluations m^d. Show how the grid size explodes exponentially. Add a horizontal dashed line at 100,000 (the fixed budget).
2. **Plot 2:** x-axis = dimension d, y-axis = absolute error (log scale). Plot both grid errors and Monte Carlo errors. Show that MC error remains stable while grid error grows.

Part (c): Write a 4-6 sentence explanation addressing:

- Why does the grid method fail in high dimensions?
- Why does Monte Carlo succeed?
- What is the practical implication for real high-dimensional problems?

HINTS

- **Q1:** Use `np.random.choice()` to select random directions. Use `np.sqrt()` to compute distances.
- **Q2:** Use `np.sin()` and logical indexing to count crossings. For the convergence plot, update the estimate at each increment.
- **Q3:** Use `np.log()` to generate decay times. Use comparison operators like `decay_times > t` to count remaining nuclei.

- **Q4:** For the grid, use `np.meshgrid()` for $d \leq 3$ or `itertools.product()` for larger d. For Monte Carlo, use vectorized operations: `X = np.random.rand(N, d); f = np.exp(-X.sum(axis=1))`.